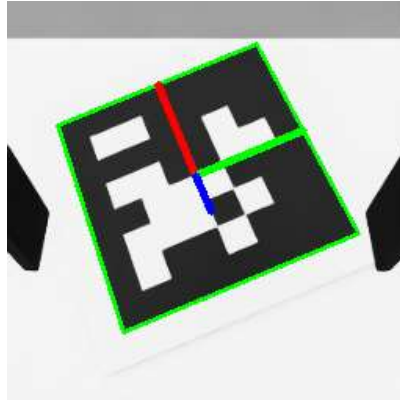


Part 1- Hand-Eye Calibration

Q1- AprilTag Pose Estimation



Q2- Hand-Eye Calibration

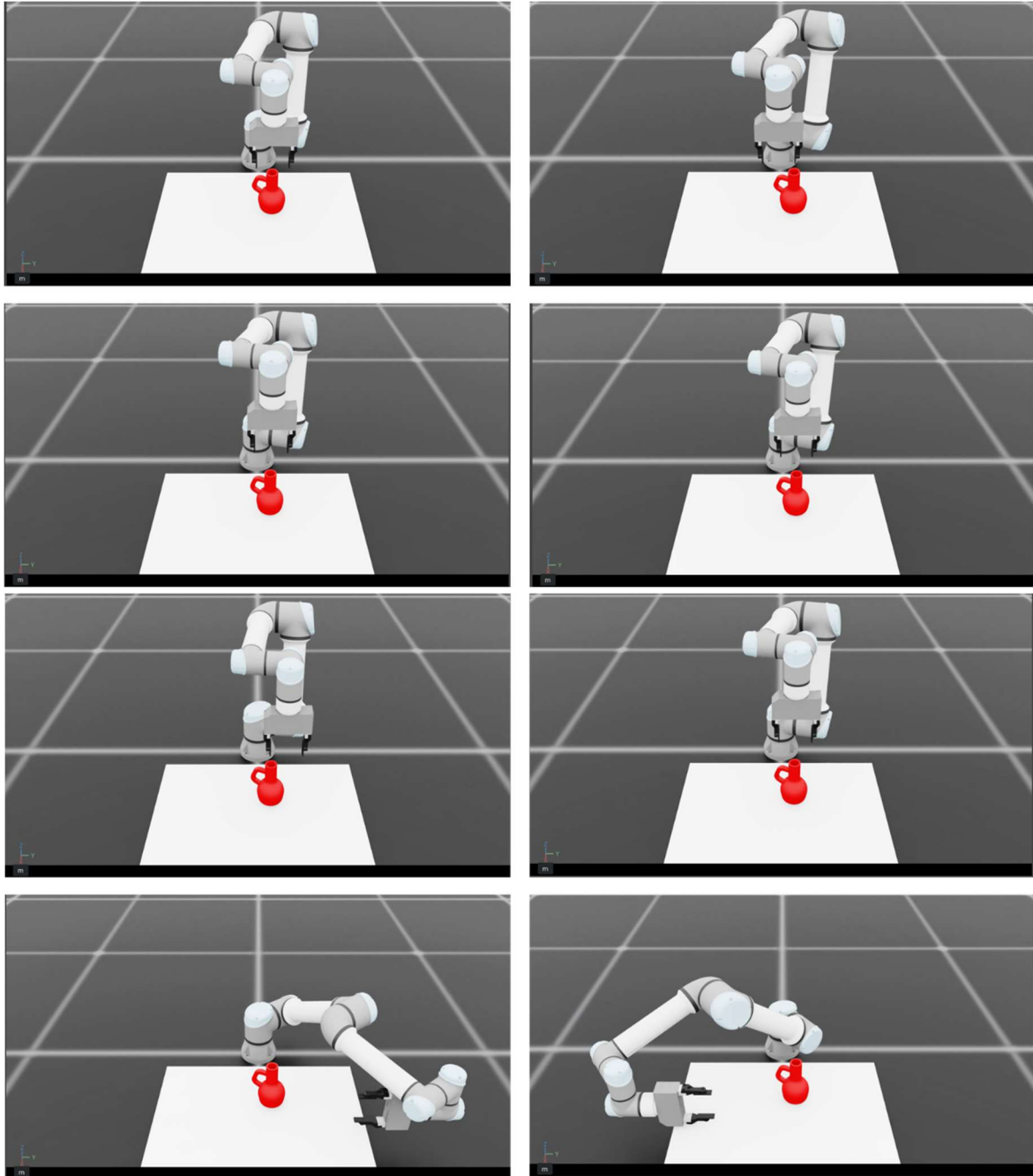
Hand-eye transformation matrix:

```
Setup complete...
T_eef_cam (translation): [[ 0.99322091 -0.09897766  0.06095618 -0.01692032]
 [ 0.11609053  0.87137651 -0.47668226  0.04783153]
 [-0.00593489  0.48052722  0.87695973 -0.08223155]
 [ 0.          0.          0.          1.          ]]
```

My goal was to capture the AprilTag from many distinct angles, including different tilts, rolls, and pitches. My script was specifically set to collect 15 of these poses, I chose to collect this many data points to ensure robustness. Every measurement I take has small, inevitable errors (noise). By using a large set of 15 poses, I created an overdetermined system. This allows the solver to find a "best-fit" solution that averages out and minimizes the impact of these random errors, leading to a much more stable and reliable calibration result than if I had used only a few.

Part 2- Pose Estimation

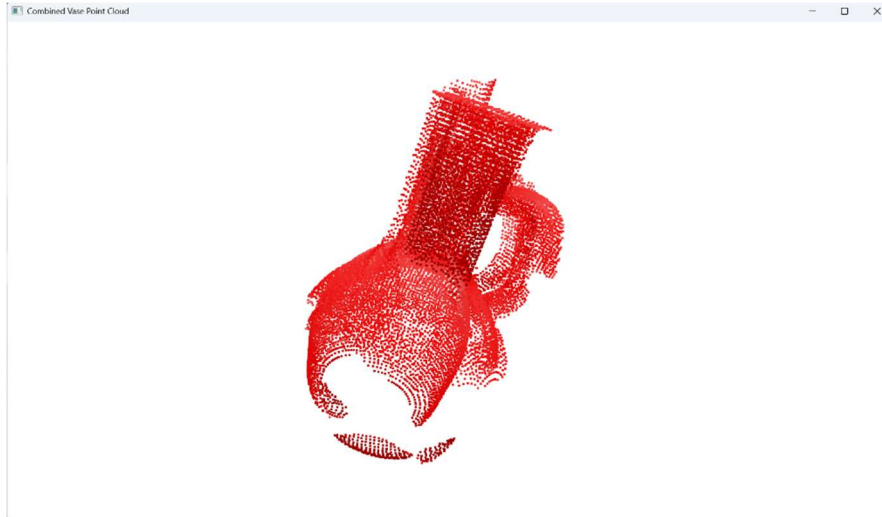
Q3- Object Point Cloud Reconstruction



I defined a fixed list of 8 target poses in the code. My strategy was to create a set of views that would capture the vase's full 360-degree shape, ensuring the final reconstructed point cloud would be as complete as possible. All these poses were calculated relative to the vase's known initial position, which I retrieved from the scene configuration. To achieve this full coverage, I programmed a combination of three different types of views. First, I included a *top-down center* view to get a clear look at the top of the vase. Next, I added four *offset top-down* views, shifting the camera to the front, back, left, and right while keeping the same downward orientation, to capture the outer top edges. Finally, to see the sides of the vase (which are

hidden from a top-down view), I added two *low-angled* poses (left and right). These poses were set at a lower height and used specific tilted orientations to look "up" at the vase's profile. By combining these 8 pre-defined poses, I could gather point cloud data from all critical angles, minimizing any gaps or occlusions in the final 3D model.

Combined vase point cloud:



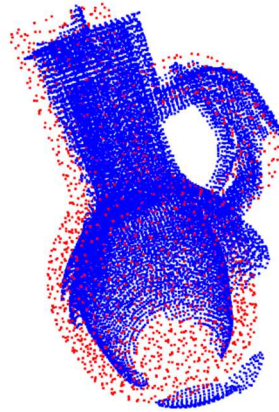
Q4- Iterative Closest Point

The Iterative Closest Point (ICP) algorithm is utilized to precisely register a canonical model of the vase against the experimentally captured data, thereby determining the vase's 6D pose in the world coordinate frame. This registration process aligns We have two distinct point cloud datasets: the source cloud from *vase.npz*, and the target cloud, which is the *combined_pcd* reconstructed by merging the 8 individual views captured by the robot's camera.

The ICP process is initialized with an initial transform that serves as an approximate estimate of the vase's location, derived from its known spawn position in the simulation. Following this initialization, the algorithm "iteratively" refines this pose. The algorithm repeatedly executes a two-step process: First, it establishes correspondences by finding the closest point in the target scene for every point in the source model; Second, it computes the optimal 4x4 rigid transformation (rotation and translation) that minimizes the least-squares distance between these corresponding point pairs.

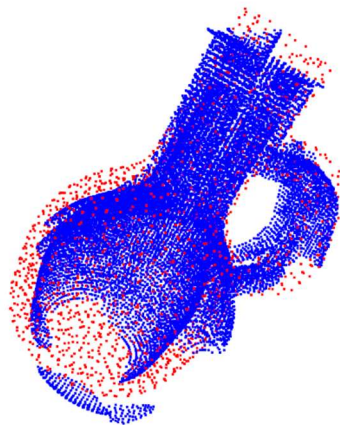
This loop continues until the alignment converges or the maximum iteration limit is reached. The final output matrix is saved as T_{world_vase} , providing the definitive pose of the vase, which is subsequently used to calculate the robot's grasping trajectory.

Before ICP (Red=Source, Blue=Target)



Before ICP(Red = Source, Blue = Target)

After ICP (Red=Source, Blue=Target)



After ICP(Red = Source, Blue = Target)

Q5- Transform Grasp Pose

The former ICP registration result is the pose of the vase in the world frame: T_{world_vase} . It's the 4x4 matrix representing the position and orientation of the vase's origin relative to the world's origin.

The file `T_vase_grasp.npz` defines the grasp pose relative to the vase's own local frame: T_{vase_grasp} . We can use matrix multiplication to transform local pose into global world pose.

$$T_{world_grasp} = T_{world_vase} \times T_{vase_grasp}$$